In [1]:

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV, learning_cur
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)


df = pd.read_csv("HeartDiseaseTrain-Test.csv")

print("Original dataset shape:", df.shape)

num_duplicates = df.duplicated().sum()
print("Number of duplicated rows:", num_duplicates)

df = df.drop_duplicates()

df = df.reset_index(drop=True)

print("Dataset shape after removing duplicates:", df.shape)
print("\nFirst five rows:")
print(df.head())
print("\nMissing values in each column:")
print(df.isnull().sum())
print("\nTarget distribution:")
print(df["target"].value_counts())
print("\nTarget distribution percentage:")
print(df["target"].value_counts(normalize=True))



plt.figure(figsize=(6, 4))
df["target"].value_counts().plot(kind="bar")
plt.title("Distribution of Target Variable")
plt.xlabel("Target")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()

numerical_features = [
    "age",
    "resting_blood_pressure",
```

```
    "cholestoral",
    "Max_heart_rate",
    "oldpeak"
]

df[numerical_features].hist(figsize=(12, 8), bins=20)
plt.suptitle("Distributions of Numerical Features")
plt.show()

plt.figure(figsize=(10, 8))
corr = df.corr(numeric_only=True)
plt.imshow(corr, aspect="auto")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Matrix")
plt.show()


X = df.drop("target", axis=1)
y = df["target"]

categorical_features = [
    "sex",
    "chest_pain_type",
    "fasting_blood_sugar",
    "rest_ecg",
    "exercise_induced_angina",
    "slope",
    "vessels_colored_by_flourosopy",
    "thalassemia"
]



preprocessor_scaled = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

preprocessor_tree = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numerical_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)



X_train_val, X_test, y_train_val, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
```

```python
    X_train_val,
    y_train_val,
    test_size=0.25,
    random_state=42,
    stratify=y_train_val
)

print("\nTraining set shape:", X_train.shape)
print("Validation set shape:", X_val.shape)
print("Test set shape:", X_test.shape)


def evaluate_model(model, X_test, y_test, model_name):
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print("\n" + "=" * 60)
    print(model_name)
    print("=" * 60)
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1-score :", round(f1, 4))

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
    plt.title(f"Confusion Matrix - {model_name}")
    plt.show()

    return {
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    }



lr_baseline = Pipeline(steps=[
    ("preprocessor", preprocessor_scaled),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])

lr_baseline.fit(X_train, y_train)

lr_val_pred = lr_baseline.predict(X_val)

print("\nLogistic Regression Baseline Validation Results")
print("Accuracy :", round(accuracy_score(y_val, lr_val_pred), 4))
print("Precision:", round(precision_score(y_val, lr_val_pred), 4))
```

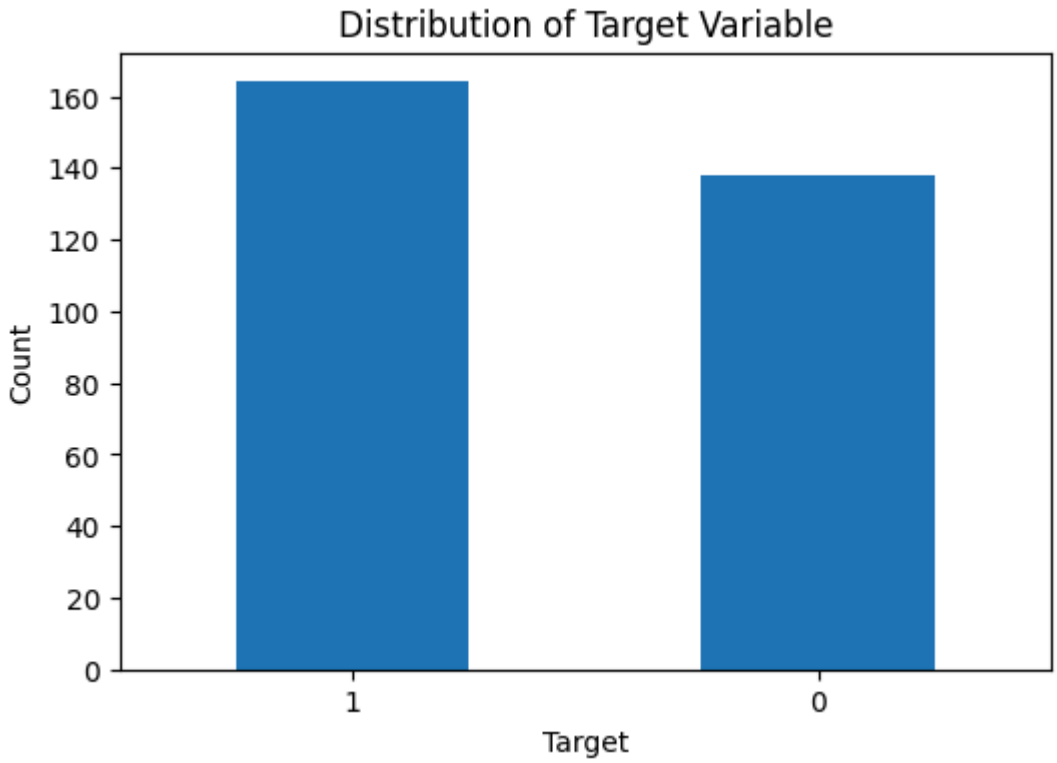
```python
print("Recall   :", round(recall_score(y_val, lr_val_pred), 4))
print("F1-score :", round(f1_score(y_val, lr_val_pred), 4))



lr_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor_scaled),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])

lr_param_grid = {
    "classifier__C": [0.01, 0.1, 1, 10, 100],
    "classifier__solver": ["lbfgs"]
}

lr_grid = GridSearchCV(
    lr_pipeline,
    lr_param_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

lr_grid.fit(X_train_val, y_train_val)

print("\nBest Logistic Regression Parameters:")
print(lr_grid.best_params_)
print("Best Logistic Regression CV F1-score:")
print(round(lr_grid.best_score_, 4))

best_lr_model = lr_grid.best_estimator_


dt_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor_tree),
    ("classifier", DecisionTreeClassifier(random_state=42))
])

dt_param_grid = {
    "classifier__max_depth": [2, 3, 4],
    "classifier__min_samples_split": [10, 20, 30],
    "classifier__min_samples_leaf": [5, 10, 15],
    "classifier__criterion": ["gini", "entropy"]
}

dt_grid = GridSearchCV(
    dt_pipeline,
    dt_param_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

dt_grid.fit(X_train_val, y_train_val)

print("\nBest Decision Tree Parameters after controlling overfitting:")
print(dt_grid.best_params_)

print("Best Decision Tree CV F1-score:")
print(round(dt_grid.best_score_, 4))
```

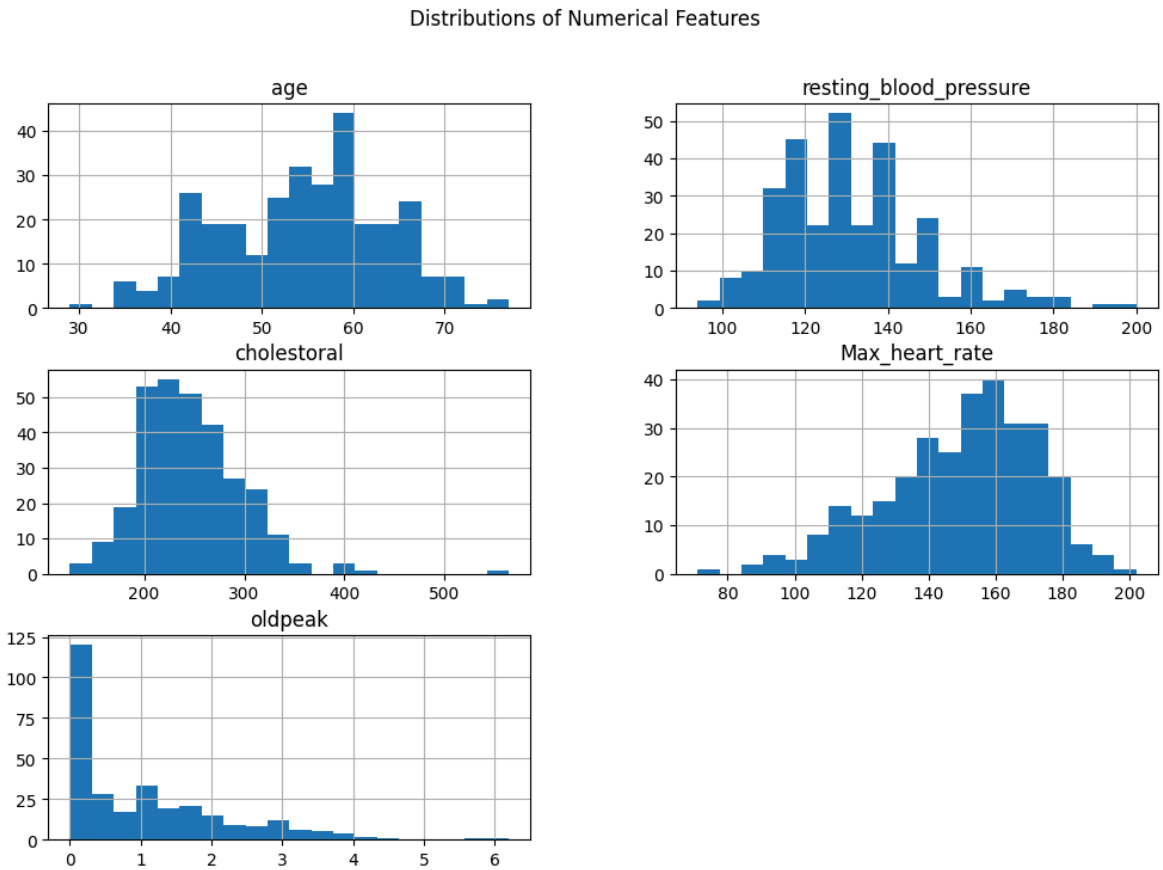
```
best_dt_model = dt_grid.best_estimator_

mlp_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor_scaled),
    ("classifier", MLPClassifier(
        max_iter=1000,
        random_state=42,
        early_stopping=True
    ))
])

mlp_param_grid = {
    "classifier__hidden_layer_sizes": [(8,), (16,), (16, 8), (32,)],
    "classifier__activation": ["relu", "tanh"],
    "classifier__alpha": [0.001, 0.01, 0.1],
    "classifier__learning_rate_init": [0.001]
}

mlp_grid = GridSearchCV(
    mlp_pipeline,
    mlp_param_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

mlp_grid.fit(X_train_val, y_train_val)

print("\nBest MLP Parameters:")
print(mlp_grid.best_params_)
print("Best MLP CV F1-score:")
print(round(mlp_grid.best_score_, 4))

best_mlp_model = mlp_grid.best_estimator_


def evaluate_train_validation(model, X_train, y_train, X_val, y_val, model_name)
    train_pred = model.predict(X_train)
    val_pred = model.predict(X_val)

    print("\n" + "=" * 60)
    print(model_name + " Train vs Validation Performance")
    print("=" * 60)

    print("Training Accuracy :", round(accuracy_score(y_train, train_pred), 4))
    print("Validation Accuracy:", round(accuracy_score(y_val, val_pred), 4))
    print("Training F1-score :", round(f1_score(y_train, train_pred), 4))
    print("Validation F1-score:", round(f1_score(y_val, val_pred), 4))


evaluate_train_validation(best_lr_model, X_train, y_train, X_val, y_val, "Logist
evaluate_train_validation(best_dt_model, X_train, y_train, X_val, y_val, "Decisi
evaluate_train_validation(best_mlp_model, X_train, y_train, X_val, y_val, "MLP")


results = []

results.append(evaluate_model(best_lr_model, X_test, y_test, "Logistic Regressio
results.append(evaluate_model(best_dt_model, X_test, y_test, "Decision Tree"))
```

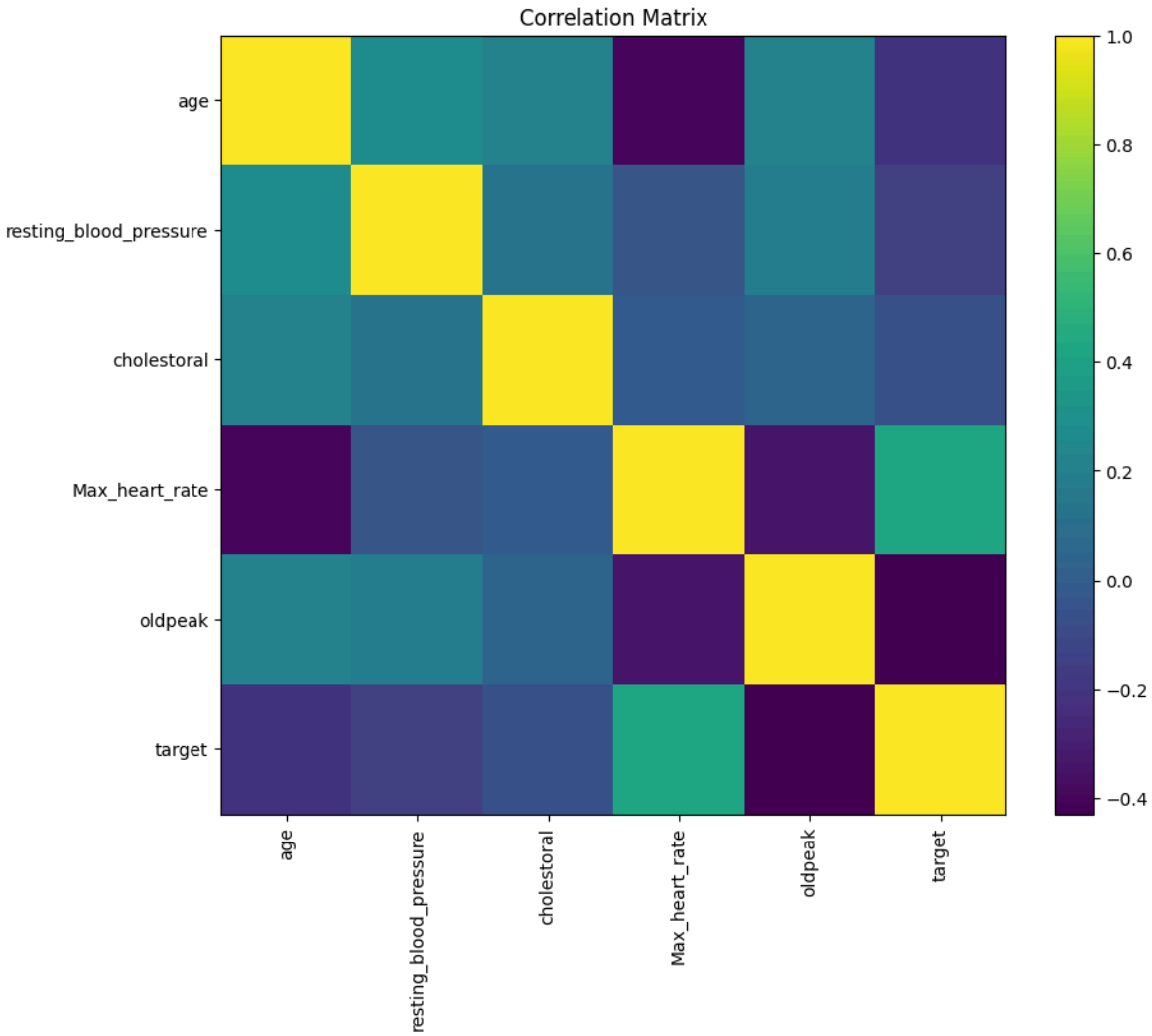
```
results.append(evaluate_model(best_mlp_model, X_test, y_test, "Multi-Layer Perce

results_df = pd.DataFrame(results)

print("\nFinal Model Comparison:")
print(results_df)


metrics = ["Accuracy", "Precision", "Recall", "F1-score"]

results_df.set_index("Model")[metrics].plot(kind="bar", figsize=(10, 6))
plt.title("Final Test Performance Comparison")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=20)
plt.legend(loc="lower right")
plt.show()

def plot_learning_curve(model, X, y, model_name):
    train_sizes, train_scores, val_scores = learning_curve(
        model,
        X,
        y,
        cv=5,
        scoring="accuracy",
        train_sizes=np.linspace(0.1, 1.0, 5),
        n_jobs=-1
    )

    train_mean = np.mean(train_scores, axis=1)
    val_mean = np.mean(val_scores, axis=1)

    train_error = 1 - train_mean
    val_error = 1 - val_mean

    plt.figure(figsize=(7, 5))
    plt.plot(train_sizes, train_error, marker="o", label="Training Error")
    plt.plot(train_sizes, val_error, marker="o", label="Validation Error")
    plt.title(f"Training vs Validation Error - {model_name}")
    plt.xlabel("Training Set Size")
    plt.ylabel("Error")
    plt.legend()
    plt.grid(True)
    plt.show()


plot_learning_curve(best_lr_model, X_train_val, y_train_val, "Logistic Regressio
plot_learning_curve(best_dt_model, X_train_val, y_train_val, "Decision Tree")
plot_learning_curve(best_mlp_model, X_train_val, y_train_val, "MLP")
```

```
Original dataset shape: (1025, 14)
Number of duplicated rows: 723
Dataset shape after removing duplicates: (302, 14)

First five rows:
   age     sex chest_pain_type  resting_blood_pressure  cholestoral  \
0   52    Male  Typical angina                     125          212
1   53    Male  Typical angina                     140          203
2   70    Male  Typical angina                     145          174
3   61    Male  Typical angina                     148          203
4   62  Female  Typical angina                     138          294

      fasting_blood_sugar               rest_ecg  Max_heart_rate  \
0    Lower than 120 mg/ml  ST-T wave abnormality             168
1  Greater than 120 mg/ml                 Normal             155
2    Lower than 120 mg/ml  ST-T wave abnormality             125
3    Lower than 120 mg/ml  ST-T wave abnormality             161
4  Greater than 120 mg/ml  ST-T wave abnormality             106

  exercise_induced_angina  oldpeak        slope vessels_colored_by_flourosopy  \
0                      No      1.0  Downsloping                           Two
1                     Yes      3.1    Upsloping                          Zero
2                     Yes      2.6    Upsloping                          Zero
3                      No      0.0  Downsloping                           One
4                      No      1.9         Flat                         Three

         thalassemia  target
0  Reversable Defect       0
1  Reversable Defect       0
2  Reversable Defect       0
3  Reversable Defect       0
4       Fixed Defect       0

Missing values in each column:
age                             0
sex                             0
chest_pain_type                 0
resting_blood_pressure          0
cholestoral                     0
fasting_blood_sugar             0
rest_ecg                        0
Max_heart_rate                  0
exercise_induced_angina         0
oldpeak                         0
slope                           0
vessels_colored_by_flourosopy   0
thalassemia                     0
target                          0
dtype: int64

Target distribution:
target
1    164
0    138
Name: count, dtype: int64

Target distribution percentage:
target
1    0.543046
```

```
0    0.456954
Name: proportion, dtype: float64
```

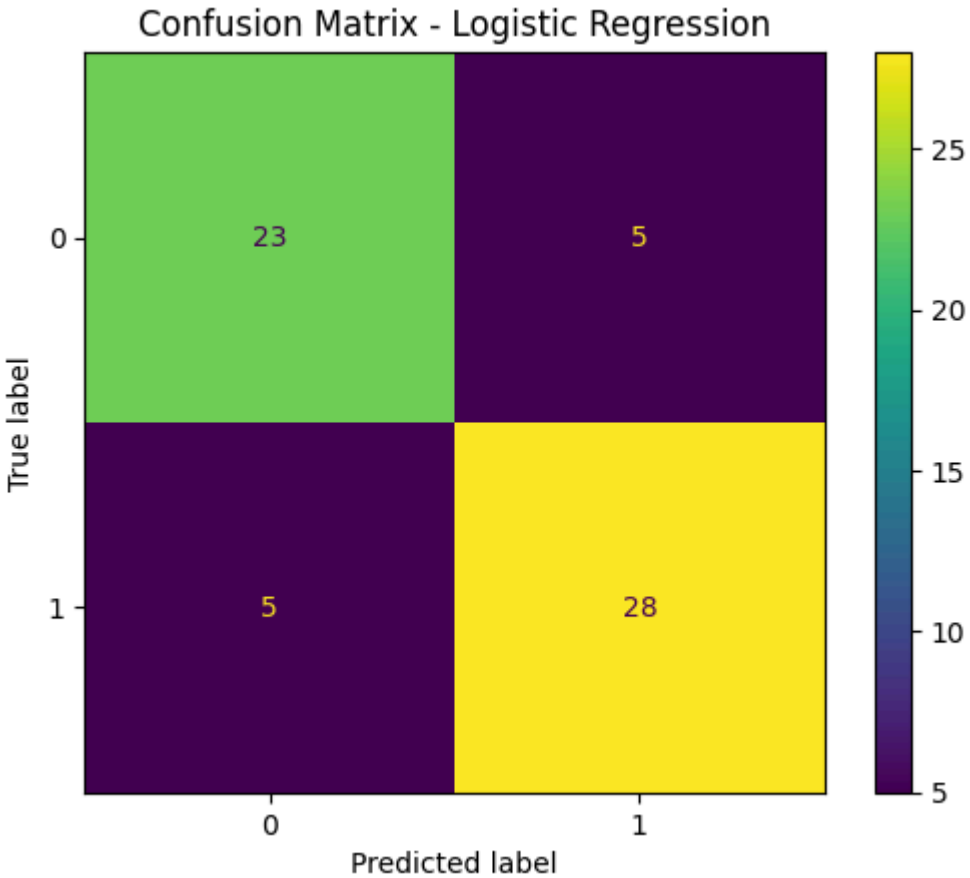




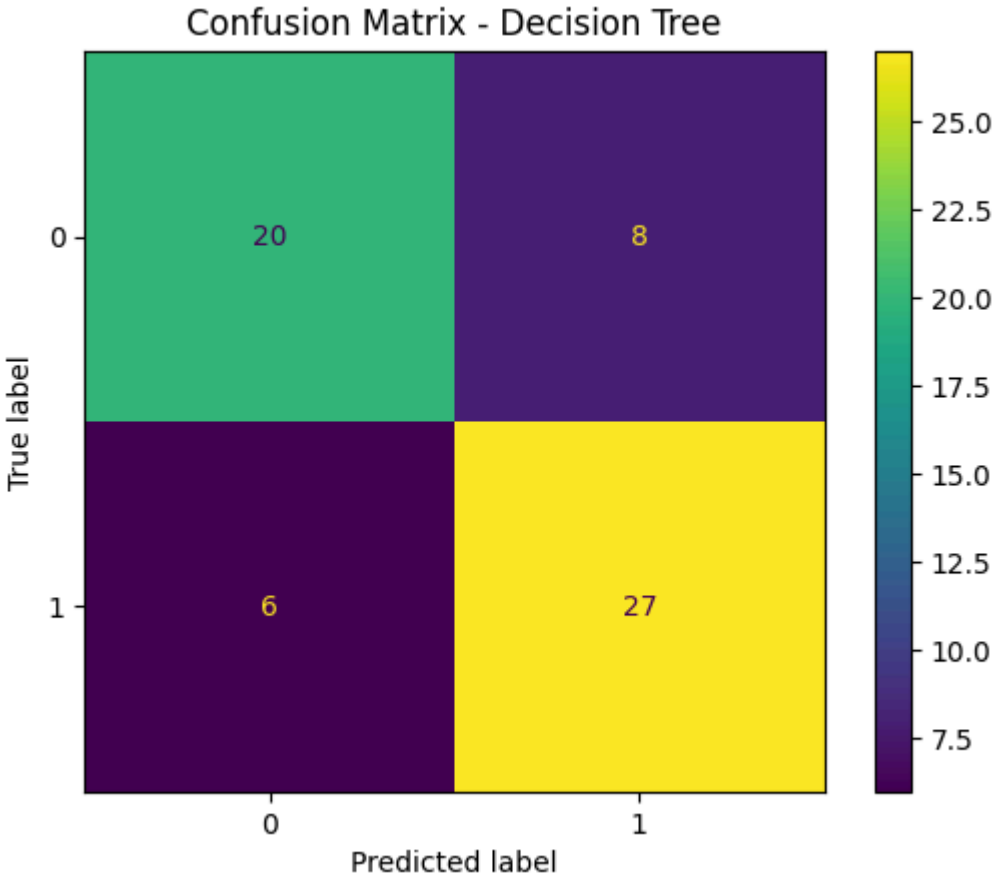

Correlation Matrix
age
resting_blood_pressure
cholestoral
Max_heart_rate
oldpeak
target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

```
Training set shape: (180, 13)
Validation set shape: (61, 13)
Test set shape: (61, 13)

Logistic Regression Baseline Validation Results
Accuracy : 0.7377
Precision: 0.7297
Recall   : 0.8182
F1-score : 0.7714

Best Logistic Regression Parameters:
{'classifier__C': 0.1, 'classifier__solver': 'lbfgs'}
Best Logistic Regression CV F1-score:
0.8617

Best Decision Tree Parameters after controlling overfitting:
{'classifier__criterion': 'gini', 'classifier__max_depth': 3, 'classifier__min_sa
mples_leaf': 15, 'classifier__min_samples_split': 10}
Best Decision Tree CV F1-score:
0.8175

Best MLP Parameters:
{'classifier__activation': 'tanh', 'classifier__alpha': 0.001, 'classifier__hidde
n_layer_sizes': (16,), 'classifier__learning_rate_init': 0.001}
Best MLP CV F1-score:
0.7795

============================================================
Logistic Regression Train vs Validation Performance
============================================================
Training Accuracy : 0.9222
Validation Accuracy: 0.7705
Training F1-score : 0.9307
Validation F1-score: 0.8

============================================================
Decision Tree Train vs Validation Performance
============================================================
Training Accuracy : 0.8722
Validation Accuracy: 0.7705
Training F1-score : 0.8832
Validation F1-score: 0.7941

============================================================
MLP Train vs Validation Performance
============================================================
Training Accuracy : 0.8111
Validation Accuracy: 0.7869
Training F1-score : 0.8211
Validation F1-score: 0.806

============================================================
Logistic Regression
============================================================
Accuracy : 0.8361
Precision: 0.8485
Recall   : 0.8485
F1-score : 0.8485

Confusion Matrix:
```

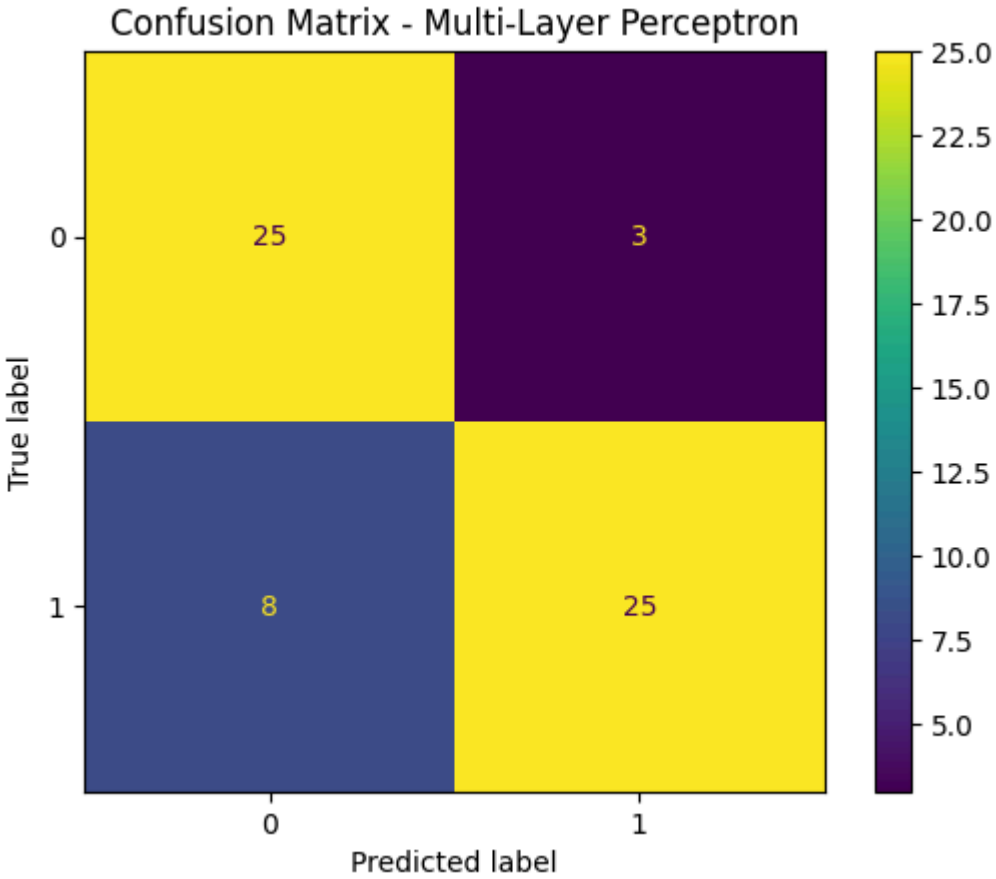
```
[[23  5]
 [ 5 28]]

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.82      0.82        28
           1       0.85      0.85      0.85        33

    accuracy                           0.84        61
   macro avg       0.83      0.83      0.83        61
weighted avg       0.84      0.84      0.84        61
```



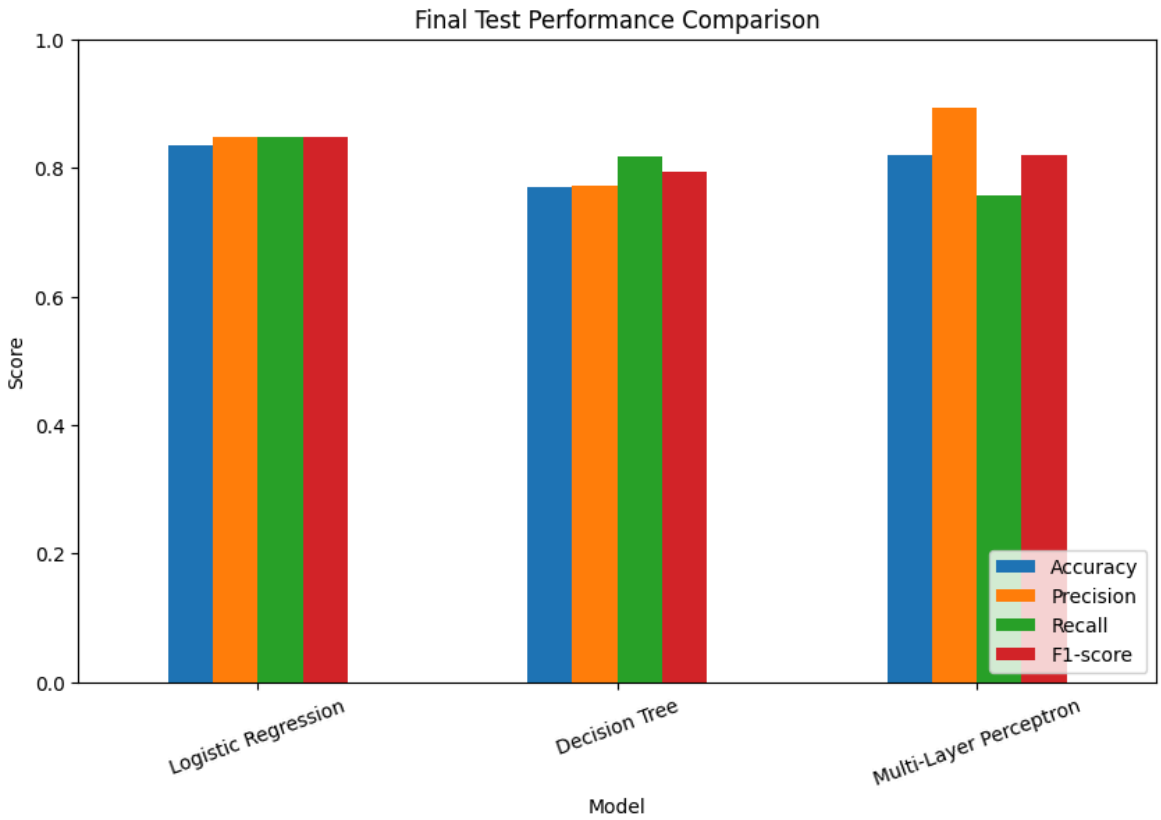
```
============================================================
Decision Tree
============================================================
Accuracy : 0.7705
Precision: 0.7714
Recall   : 0.8182
F1-score : 0.7941

Confusion Matrix:
[[20  8]
 [ 6 27]]

Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.71      0.74        28
           1       0.77      0.82      0.79        33

    accuracy                           0.77        61
   macro avg       0.77      0.77      0.77        61
weighted avg       0.77      0.77      0.77        61
```

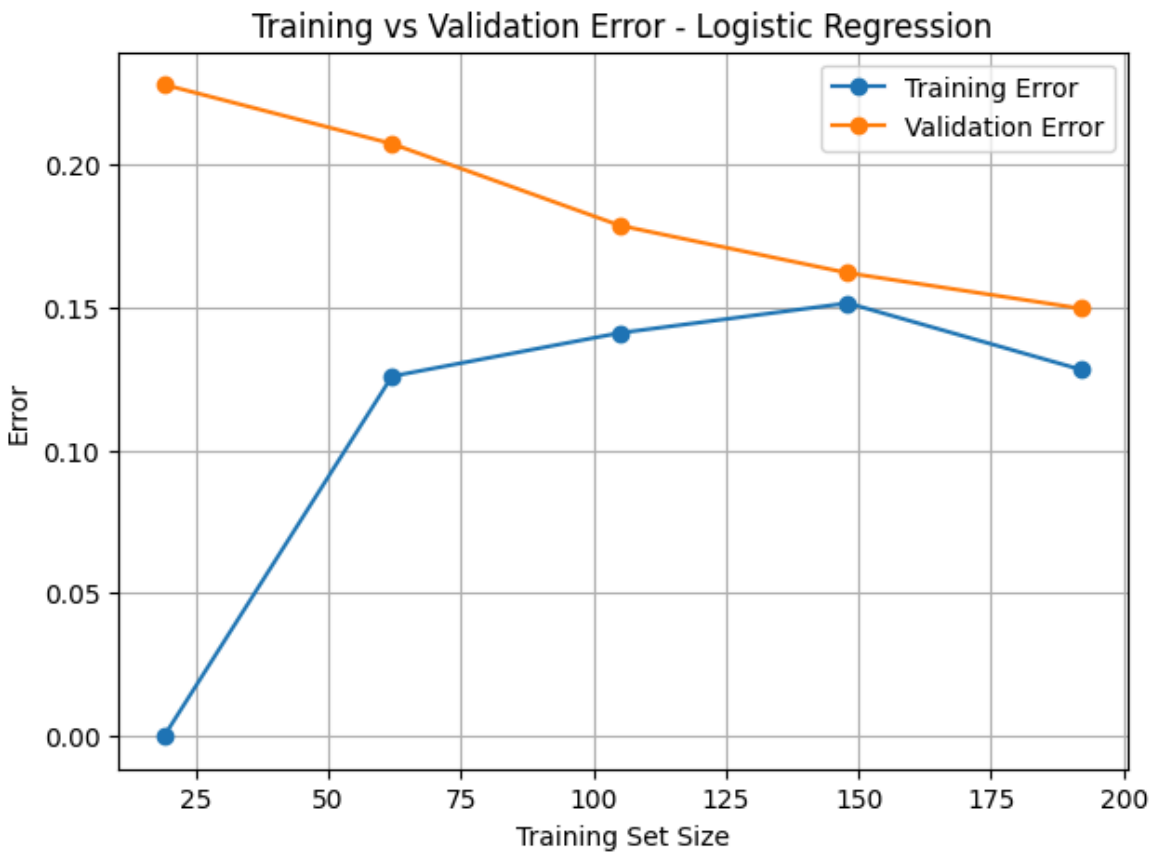


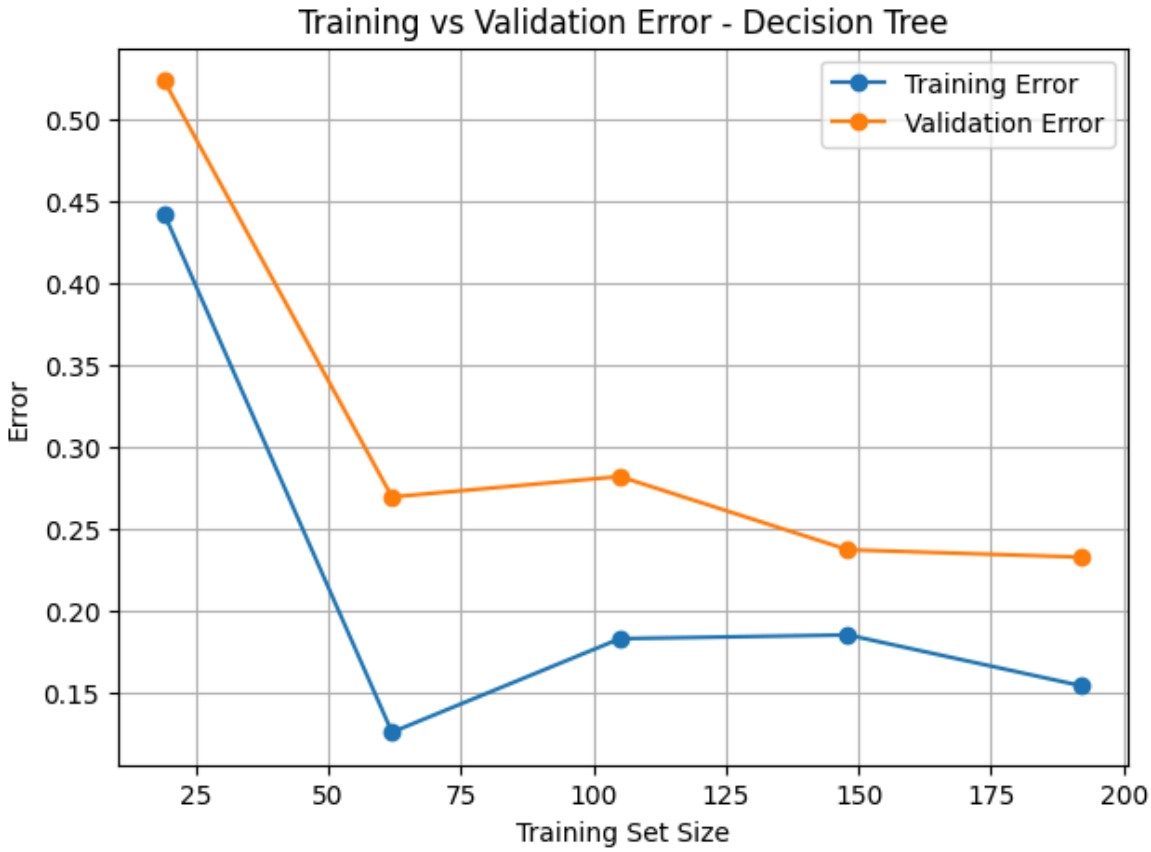
```
============================================================
Multi-Layer Perceptron
============================================================
Accuracy : 0.8197
Precision: 0.8929
Recall   : 0.7576
F1-score : 0.8197

Confusion Matrix:
[[25  3]
 [ 8 25]]

Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.89      0.82        28
           1       0.89      0.76      0.82        33

    accuracy                           0.82        61
   macro avg       0.83      0.83      0.82        61
weighted avg       0.83      0.82      0.82        61
```

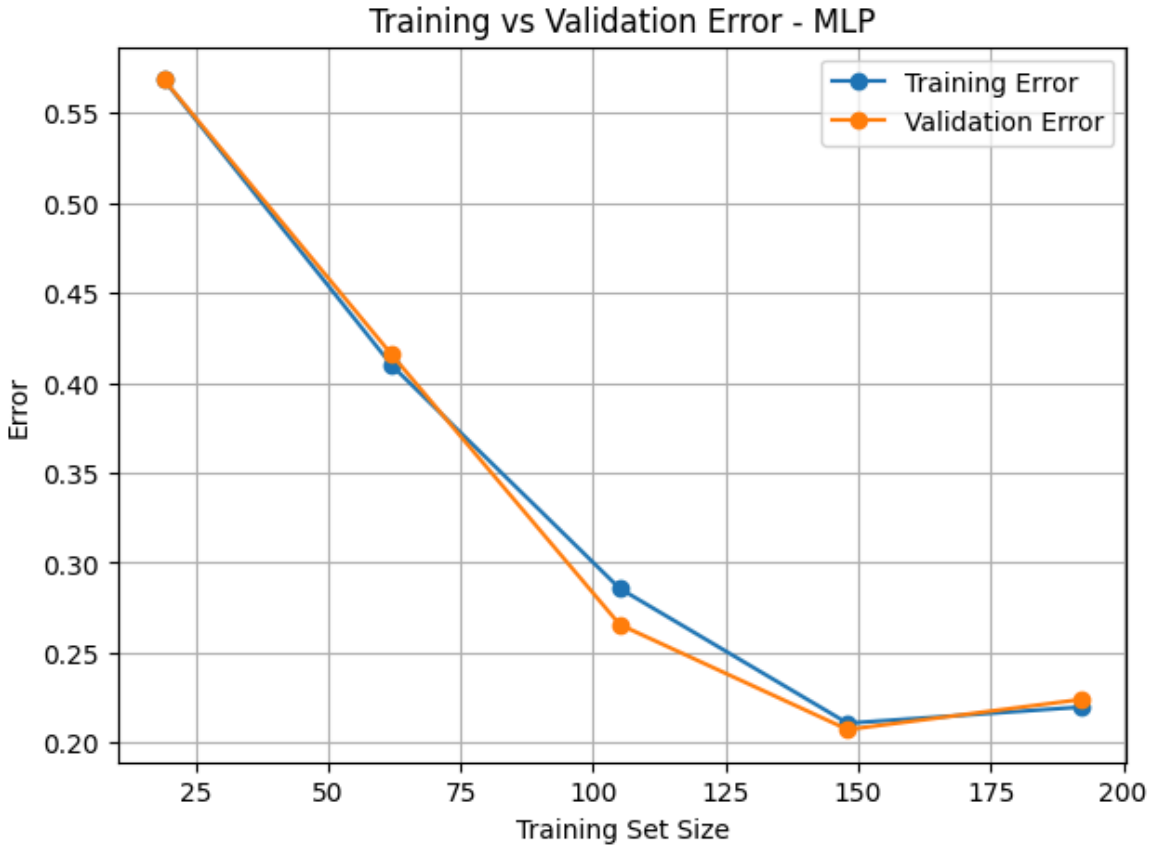



```
Final Model Comparison:
                    Model  Accuracy  Precision    Recall  F1-score
0     Logistic Regression  0.836066   0.848485  0.848485  0.848485
1           Decision Tree  0.770492   0.771429  0.818182  0.794118
2  Multi-Layer Perceptron  0.819672   0.892857  0.757576  0.819672
```


Final Test Performance Comparison
1.0
0.8
0.6
0.4
0.2
0.0
Score
Accuracy
Precision
Recall
F1-score
Logistic Regression
Decision Tree
Multi-Layer Perceptron
Model


Training vs Validation Error - Logistic Regression
Training Error
Validation Error
0.20
0.15
0.10
0.05
0.00
Error
25
50
75
100
125
150
175
200
Training Set Size


Training vs Validation Error - Decision Tree
Training Error
Validation Error
Error
0.50
0.45
0.40
0.35
0.30
0.25
0.20
0.15
25
50
75
100
125
150
175
200
Training Set Size


Training vs Validation Error - MLP
Training Error
Validation Error
Error
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.20
25
50
75
100
125
150
175
200
Training Set Size

In [ ]: